we buy ourselves some benefits. For example, we may want to extend our application in the future to support users in many countries. In that case, we'd want to localize the display of the time, choosing a time appropriate to the user's time zone. That would require a fair amount of application-level code, and it would probably not be appropriate to embed it at the view level. By setting the time to display in the controller, we make our application more flexible: we can change the time zone in the controller without having to update any view that uses that time object. The time is *data*, and it should be supplied to the view by the controller. We'll see a lot more of this when we introduce models into the equation.

The Story So Far

Let's briefly review how our current application works.

1. The user navigates to our application. In our case, we do that using a local URL such as http://localhost:3000/say/hello.

2. Rails then matches the route pattern, which it previously split into two parts and analyzed. The `say` part is taken to be the name of a controller, so Rails creates a new instance of the Ruby `SayController` class (which it finds in `app/controllers/say_controller.rb`).

3. The next part of the pattern, `hello`, identifies an action. Rails invokes a method of that name in the controller. This action method creates a new `Time` object holding the current time and tucks it away in the `@time` instance variable.

4. Rails looks for a template to display the result. It searches the `app/views` directory for a subdirectory with the same name as the controller (`say`) and in that subdirectory for a file named after the action (`hello.html.erb`).

5. Rails processes this file through the ERB templating system, executing any embedded Ruby and substituting in values set up by the controller.

6. The result is returned to the browser, and Rails finishes processing this